

Computer Programming Summer Interest Group

Session 8 / Turtle Graphics

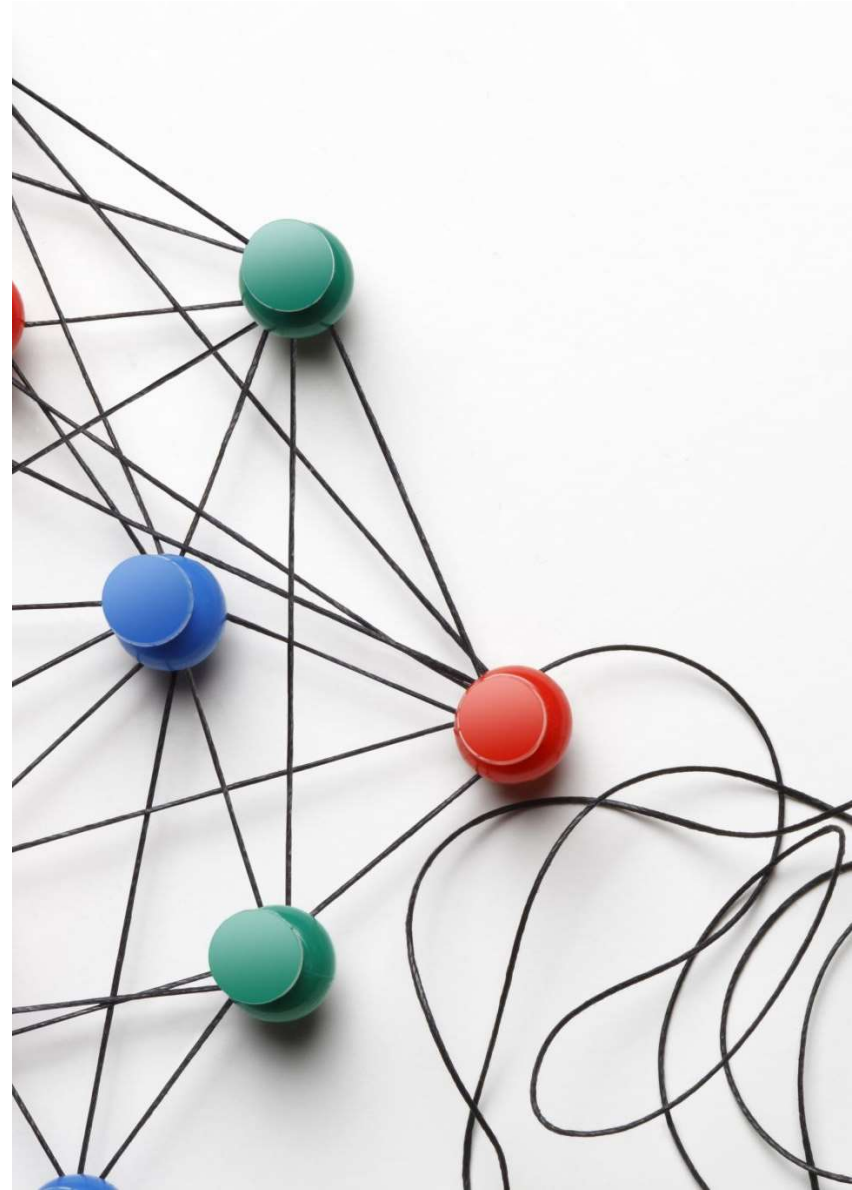
8/6/2025

Matthew 6:28-30

And why are you anxious about clothing? Consider the lilies of the field, how they grow: they neither toil nor spin, yet I tell you, even Solomon in all his glory was not arrayed like one of these. But if God so clothes the grass of the field, which today is alive and tomorrow is thrown into the oven, will he not much more clothe you, O you of little faith?

Objectives

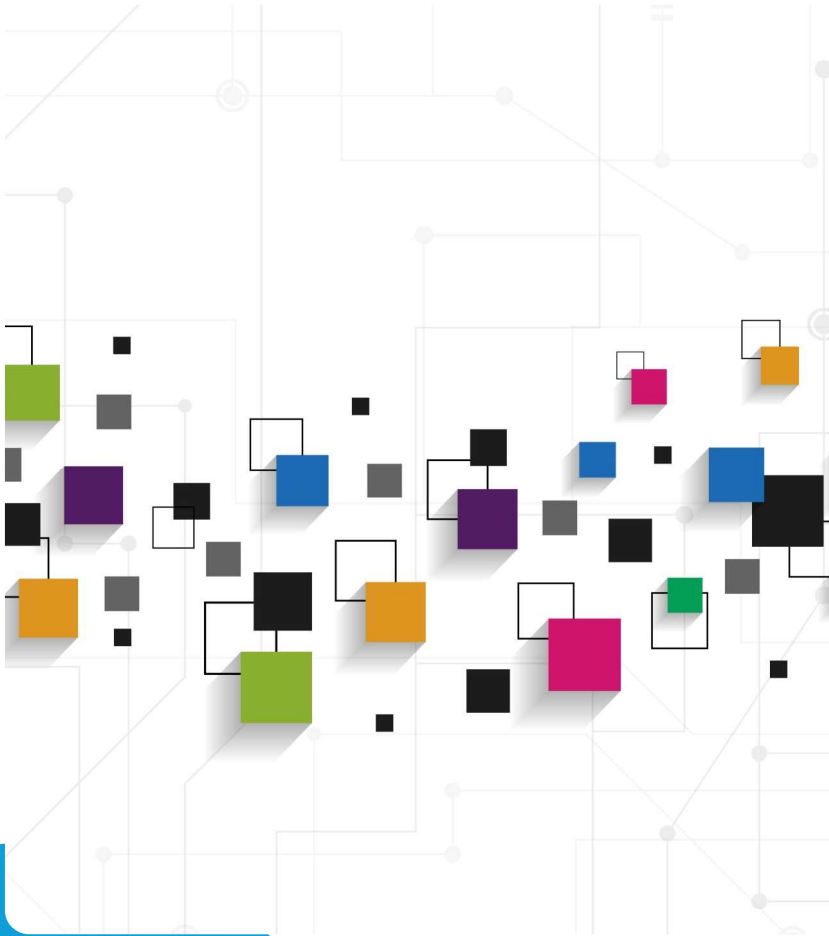
- By the end of this session, you will be able to:
 - Understand what Turtle Graphics is and why it's useful.
 - Use basic turtle commands to move and draw on the screen.
 - Create simple geometric shapes using loops.
 - Write and run Python scripts that use Turtle Graphics.





Turtle Graphics

- Derived from the [Logo programming language](#)
- Used to create simple 2D drawings.
- Mimic's the idea of drawing something by hand
- Intended to be an educational tool
- There are many online examples of source code / images made with Turtle.
- The official Python Turtle documentation describes all f the



Elements of a Turtle Graphic scene

Canvas

- The portion of the screen upon which to draw the scene
- Characteristics of the canvas (color, size) can be changed

Cursor

- Indicator of the location of the “Turtle” upon the canvas
- The shape and presence of the cursor can change

Turtle

- The pen used to draw on the canvas
- Accepts commands to draw using a pen with configurable size and color

Create a Blank Canvas

- In order to use Turtle, you need to “import” the turtle package
 - This gives your program access to the Python Turtle graphics library
- Create an instance of the turtle.Turtle class
 - Creates a canvas with a white background
- Prevent the run from immediately closing the canvas
 - `turtle.done` – ends program run when canvas window is closed
 - `turtle.exitonclick` – ends program when mouse is clicked on canvas

```
import turtle
```

```
t = turtle.Turtle()
```

```
turtle.done()
```

Drawing lines

- Draw line on canvas
 - forward – Draws a line in the current “forward” direction the specified distance
 - backward – Draws a line in the opposite of the current “forward” direction the specified distance
- Change the direction
 - left – Rotate the “forward” direction to the left by the number of degrees
 - right – Rotate the “forward” direction to the right by the number of degrees
- Change the angle units
 - degrees – Use degree units in the angle
 - radians – Use radians as the angle units

```
import turtle
import math

t = turtle.Turtle()

t.radians()
t.left(math.pi / 2.0)
t.forward(100)

t.degrees()
t.left(90)
t.forward(100)

turtle.done()
```


More examples – Drawing a Triangle

- Draw 3 sides of length 100
- Turn 120° to the left at each corner

```
import turtle

t = turtle.Turtle()
for i in range(3):
    t.forward(100)
    t.left(120)

turtle.done()
```


More examples – Draw a Square

- Draw 4 sides of length 100
- Turn 90° to the left at each corner

```
import turtle
```

```
t = turtle.Turtle()  
for i in range(4):  
    t.forward(100)  
    t.left(90)
```

```
turtle.done()
```

More examples

- Draw tangential circles

- Draw 10 circles of radius $i \cdot 10$
- Each circle touches at (0, 0)

```
import turtle

t = turtle.Turtle()
t.pensize(3)
for i in range(0, 100, 10):
    t.circle(i)

turtle.done()
```



Activities

- Draw a house
- Draw a spirograph of circles
 - Loop i over range (n) for some n
 - Set angle to $2\pi * i / n$
 - Set x to $200 * \cos(\text{angle})$
 - Set y to $200 * \sin(\text{angle})$
 - Both $\sin$ and $\cos$ are functions in the math package
 - Set turtle position to (x, y)
 - Draw a circle of radius 150